

操作系统重点

2020年1月4日 9:48

操作系统历史:

- 第一代: 真空管和穿孔卡片 (1945-1955)
- 第二代: 晶体管和批处理系统 (1955-1965)
- 第三代: 集成电路和多道程序设计 (1965-1980)
- 第四代: 个人计算机 (1980-至今)
- 第五代: 移动计算机 (1990-至今)

绪论

1. 计算机的两种运行状态

(内核态和用户态的区别和划分)

内核态和用户态, 其本质就是CPU的两种状态。

- **内核态**, 又称为管态、核心态。软件中最基本的部分是操作系统, 它运行在内核态。在这个模式中, 操作系统具体对所有硬件的完全访问权, 可以执行机器能够运行的任何指令。
- **用户态**, 又称目态。处理机运行应用程序的状态。软件的其余部分运行在用户态。在用户态下, 只是用了机器指令中的一个子集, 特别的, 会影响机器控制或可进行IO操作的指令, 在用户态中的程序里是禁止的。
- 用户接口程序 (shell或者GUI) 处于用户态程序中的最低层次, 允许用户运行其他程序, 诸如Web浏览器、电子邮件阅读器、音乐播放器等。

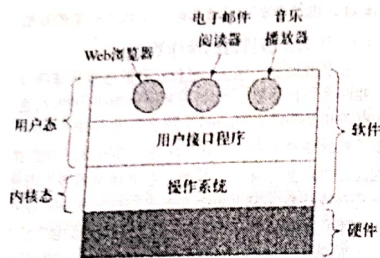


图1-1 操作系统所处的位置

- 操作系统运行在裸机之上, 为所有其他软件提供基础的运行环境。

操作系统和普通软件 (用户态) 的主要区别:

用户不可以写会影响机器控制或可进行IO操作的指令的程序。这个程序由硬件保护, 防止用户试图对其修改。

为什么要分状态:

在CPU的所有指令中, 有些指令是非常危险的, 如果错用, 将导致系统崩溃, 比如清内存、设置时钟等。如果所有的程序都能使用这些指令, 那么系统死机的概率将大大增加。所以, CPU将指令分为特权指令和非特权指令, 对于那些危险的指令, 只允许操作系统及其相关模块使用, 普通应用程序只能使用那些不会造成灾难的指令。

2. 并发、并行概念区分

- **并行**: 指在同一时刻, 有多条指令在多个处理器上同时执行。所以无论从微观还是宏观来看, 二者都是一起执行的。
- **并发**: 指在同一时刻只能有一条指令执行, 但多个进程指令被快速的轮换执行, 使得在宏观上具有多个进程同时执行的效果, 但在微观上并不是同时执行的, 只是把时间分成若干段, 使多个进程快速交替的执行。

(还有其他解释, 理解回答一下)



并发、并行

并发(多线程)

当有多 T 线程在操作时,如果系统只有一 T CPU,则它根本不可能真正同时进行一 T 以上的线程,它只能把CPU运行时间划分为各 T 时间段,再将时间段分配给各位线程执行,在一 T 时间段的线程代码运行时,其它线程处于挂起状态。——并发。

并行(多核)。

当系统有一 T 以上的CPU时,则线程的操作有可能非并发。当一 T CPU执行一 T 线程时,另一 T CPU可以执行另一 T 线程,两线程不抢占CPU资源,可以同时进行的,这种方式我们称之为并行。

区别:

并发: 指两 T 或多 T 事件在同一时刻发生; 而并行指两 T 或多 T 事件在同一时间间隔内发生。在多道程序环境下,并发是指一段时间内宏观上有多道程序在同时运行,但在单处理机系统上,每一时刻只能有一道程序执行,故微观上这些程序只能是分时地交替执行。倘若在计算机系统中有多个 T 处理机,则这些可以并发执行的程序便可以分配到多个 T 处理机上,真正实现并行执行。



- 并行是指两个或者多个事件在同一时刻发生；而并发是指两个或多个事件在同一时间间隔发生。
- 并行是在不同实体上的多个事件，并发是在同一实体上的多个事件。
- 并行是在一台处理机上“同时”处理多个任务，并发是在多个处理器上同时处理多个任务。

3. 现代操作系统的基本特征，有核心的两个特征是什么？

操作系统有四个基本特征：并发、共享、虚拟、异步。其中，并发和共享是最核心的两个特征。

并发

- √并发是指两个或多个事件在同一时间间隔内发生。操作系统的并发性是指计算机系统中同时存在多个运行着的程序，因此它应该具有处理和调度多个程序同时执行的能力。
- 在这种多道程序环境下，一段时间内，宏观上有多个程序在同时运行，而每一时刻，单处理器环境下实际仅能有一道程序执行，故微观上这些程序还是在分时地交替执行。操作系统的并发性是通过分时得以实现的。

共享

- √共享是指系统中的资源（硬件资源和信息资源）可以被多个并发执行的程序共同使用，而不是被其中一个独占。资源共享有两种方式：互斥访问和同时访问。

(并发和共享的关系，不知道会不会问)

- √并发和共享是操作系统的最基本特征，互为依存。并发执行的要求引出了资源的共享；而资源共享的管理又直接影响到程序的并发执行。

虚拟

- √虚拟性是一种管理技术，把物理上的一个实体变成逻辑上的多个对应物，或把物理上的多个实体变成逻辑上的一个对应物的技术。
- 采用虚拟技术的目的是为用户提供易于使用、方便高效的操作环境。

异步

- √异步在多道程序环境下，允许多个程序并发执行，但由于资源有限，进程的执行不是一贯到底。而是走走停停，以不可预知的速度向前推进，这就是进程的异步性。
- 异步性使得操作系统运行在一种随机的环境下，可能导致进程产生与时间有关的错误。但是只要运行环境相同，操作系统必须保证多次运行程序，都获得相同的结果。

4. 系统调用的原理

- 在计算机中，系统调用（英语：system call），指运行在用户态的程序向操作系统内核请求需要更高权限运行的服务。
- 系统调用提供了用户程序与操作系统之间的接口（即系统调用是用户程序和内核交互的接口）。
- 为了从操作系统中获得服务，用户程序必须使用系统调用，以陷入内核并调用操作



系统。当有关工作完成之后，把控制返回给在系统调用后面跟随着的指令。

简单理解：应用程序有时会需要一些危险的、权限很高的指令，如果把这些权限放心地交给用户程序是很危险的(比如一个进程可能修改另一个进程的内存区，导致其不能运行)，但是又不能完全不给这些权限。于是有了系统调用，危险的指令被包装成系统调用，用户程序只能调用而无权自己运行那些危险的指令。另外，计算机硬件的资源是有限的，为了更好的管理这些资源，所有的资源都由操作系统控制，进程只能向操作系统请求这些资源。操作系统是这些资源的唯一入口，这个入口就是系统调用。

系统调用过程

- a. 把参数压入堆栈
- b. 对库过程的实际调用
- c. 把系统调用的编号放在操作系统所期望的地方，如寄存器
- d. 执行trap指令，将用户态切换到内核态，并在内核的一个固定地址开始执行
- e. 跟随在trap指令后的内核代码开始检查系统调用编号，然后分派给正确的系统调用处理器
- f. 系统调用处理器执行
- g. 系统调用处理器完成工作，控制返回给用户空间库过程
- h. 返回到用户程序
- i. 用户程序清除堆栈

TRAP指令

- o 把用户态切换成内核态，并启用操作系统，在内核中一个固定地址开始执行。
- o TRAP指令与过程调用非常类似，后面都跟随着一个来自远处位置的指令，以及供以后使用的一个保存在栈中的返回地址。
- o 与过程指令的两个差别：
 - TRAP的副作用是，切换到内核态。而过程调用指令并不改变模式。
 - 不像给定过程所在的相对或绝对地址那样，TRAP指令不能跳转到任意地址上。

系统调用的类型

- o 用于进程管理的系统调用
- o 用于文件管理的系统调用
- o 用于设备管理的系统调用
- o 用于套接字(socket)的系统调用
- o 用于信息维护的系统调用

系统调用的例子



UNIX	Win32	说明
fork	CreateProcess	创建一个新进程
waitpid	WaitForSingleObject	等待一个进程退出
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	终止执行
open	CreateFile	创建一个文件或打开一个已有的文件
close	CloseHandle	关闭一个文件
read	ReadFile	从一个文件读数据
write	WriteFile	把数据写入一个文件
lseek	SetFilePointer	移动文件指针
stat	GetFileAttributesEx	取得文件的属性
mkdir	CreateDirectory	创建一个新目录
rmdir	RemoveDirectory	删除一个空目录
link	(none)	Win32不支持link
unlink	DeleteFile	销毁一个已有的文件
mount	(none)	Win32不支持mount
umount	(none)	Win32不支持umount
chdir	SetCurrentDirectory	改变当前工作目录
chmod	(none)	Win32不支持安全性 (但NT支持)
kill	(none)	Win32不支持信号
time	GetLocalTime	获得当前时间

图1-23 与图1-18中UNIX调用大致对应的Win32 API调用



Quiz

- Which of the following several instructions should be executed only in kernel mode?
 - A. mask all interrupts
 - B. read current date
 - C. set current date
 - D. write the image core
 - E. read memory in user address space
 - F. halt

5. 在操作系统中，不可被中断的操作叫什么？

✓ 在操作系统中，不可被中断的操作叫原语。

- 原语PRIMITIVE是由若干条指令所组成的一个指令序列，用来实现某个特定的操作功能。这个指令序列的执行是连续的，具有不可分割性，在执行时也不可间断，直至该指令序列执行结束
- 原子操作 (atomic operation)：一个操作中的所有动作，要么全做，要么全不做
- 原语是os核心(由一组程序模块所组成的、完成os中基本功能)的一个组成部分，原语必须在内核态下执行，并且常驻内存

6. 原语、系统调用、临界区的区别——三者概念 + 区别

- 原语具有不可中断性，它是通过在其执行过程中关闭中断实现的。原语往往是被系统进程所调用。

- 临界区：把对临界资源进行访问的程序片段称作临界区域或临界区。

67. ✓ 7. 操作系统的基本管理功能有哪些？

1. 进程管理
2. 内存管理
3. 设备管理
4. 文件系统管理
5. 提供用户接口

区别：原语和系统调用是不同的概念。原语强调操作的不可分割性，可以认为是一不可中断的。系统调用，但是系统调用是用户态进入到内核态，虽然系统调用一般也会被中断，但如果有更高更紧迫的系统调用，还是能打断原来系统调用的。



- CLI

- GUI

- API

- 6. 任务管理

8. 假脱机技术

(描述假脱机技术及其组成)

基本含义

- **SPOOLing技术**是低速输入输出设备与主机交换的一种技术，通常也称为“假脱机真联机”，他的核心思想是以联机的方式得到脱机的效果。
- 低速设备经通道和设在主机内存的缓冲存储器与高速设备相联，该高速设备通常是辅存。为了存放从低速设备上输入的信息，或者存放将要输出到低速设备上的信息（来自内存），在辅存分别开辟一固定区域，叫“输出井”（对输出），或者“输入井”（对输入）。
- **√简单来说**：在内存中形成缓冲区，在高级设备形成输出井和输入井，传递的时候，从低速设备传入缓冲区，再传到高速设备的输入井，（如果要输出显示）再从高速设备的输出井，传到缓冲区，再传到低速设备。

组成

- **输入井和输出井**：输入井和输出井的存储区域是在磁盘上开辟出来的。输入输出井中的数据一般以文件的形式组织管理，这些文件称之为井文件。一个文件仅存放某一个进程的输入或输出数据，所有进程的数据输入或输出文件链接成为一个输入输出队列。
- **输入缓冲区和输出缓冲区**：输入缓冲区和输出缓冲区的存储区域是在内存中开辟出来的。主要用于缓和CPU和磁盘之间速度不匹配的矛盾。输入缓冲区用于暂存有输入设备传送的数据，之后再传送到输入井；输出缓冲区同理。
- **输入进程和输出进程**：输入进程也称为预输入进程，用于模拟脱机输入时的外围控制机，将用户要求的数据从输入设备传送到输入缓冲区，再存放到输入井。当CPU需要的时候，直接从输入井将数据读入内存。反之，输出的同理。
- **井管理程序**：用于控制作业与磁盘井之间信息的交换。

具体原理

- i. 它在输入和输出之间增加了“输入井”和“输出井”的排队转储环节，以消除用户的“联机”等待时间。
- ii. 在系统输入模块收到作业输入请求信号后，输入管理模块中的读过程负责将信息从输入装置中读入输入井缓冲区。当缓冲区满时，由写过程将信息从缓冲区写到外存的输入井中，读过程和写过程反复循环，直到一个作业输入完毕。
- iii. 当读过程读到一个硬件结束标志之后，系统再次驱动写过程把最后一批信息写入外存输入井并调用中断处理程序结束该次输入。
- iv. 然后，系统为该作业建立作业控制块，从而使输入井中的作业进入作业等待队列，等待作业调度程序选中后进入内存运行。
- v. 系统在管理输入井过程中可以“不断”读入输入的作业，直到输入结束或输入井满而暂停。



特点

- **提高了I/O的速度**: 对数据执行的I/O操作, 已从对低速I/O设备执行的I/O操作演变为对磁盘缓冲区中数据的存取, 如同脱机输入输出一样, 提高了I/O速度, 缓和了CPU和低速的I/Os设备之间速度的不匹配的矛盾。
- **将独占设备改造成了共享设备**: 因为在假脱机打印机系统中, 实际上并没有为任何进程分配设备, 而只是在磁盘缓冲区中为进程分配了一个空闲盘块和建立了一张I/O请求表。
- **实现了虚拟设备功能**: 宏观上, 对于每一个进程而言, 它们认为是自己独占了一个设备, 即使实际上是多个进程在同时使用一台独占设备。也可以说, 假脱机系统, 实现了将独占设备变换为若干台对应的逻辑设备的功能。

实例

将一台独享打印机改造为可供多个用户共享的打印机, 是应用SPOOLing技术的典型实例。

具体做法是: 系统对于用户的打印输出, 并不真正把打印机分配给该用户进程, 而是先在输出井中申请一个空闲盘块区, 并将要打印的数据送入其中; 然后为用户申请并填写请求打印表, 将该表挂到请求打印队列上。若打印机空闲, 输出程序从请求打印队首取表, 将要打印的数据从输出井传送到内存缓冲区, 再进行打印, 直到打印队列为空。

生活实例

(不知道对不对)

点外卖, 假设多个人同时点了同一家店铺的外卖, 且会由一个外卖小哥配送; 人们点完外卖的时候, 所有的订单需求都会发送到店家, 然后人们可以去去做自己的事情, 店家按照订单顺序进行制作, 制作完毕之后再由外卖小哥依次送出。

9. 列举一些操作系统

(猜的不知道考不考)

- Windows
- UNIX
- Linux
- MACOS
- Android
- IOS

多道程序设计

多道程序设计是指计算机内存中同时存放几道相互独立的程序, 使它们在管理程序控制之下, 相互穿插运行(内存中多道程序轮流地或分时占有CPU), 交叉地执行(争CPU情况), 使他们共享CPU和系统中各种资源。这种快速的切换不被称作多道程序设计。

10. 进程线程基本概念及区别

在进程模型中, 计算机上所有可运行的软件, 通常也包括操作系统, 被组织成若干顺序进程, 简称进程。

一个进程就是一个正在执行程序实例, 包括程序计数器、寄存器和变量的当前值。进程是系统进行资源分配和调度的基本单位, 是操作系统结构的基础。

从概念上说, 每个进程拥有它自己的虚拟CPU。实际上真正的CPU在各进程之间切换。进程由程序、数据以及进程控制块组成。

1. 进程因为等待输入不被阻塞
2. 调度程序选择另一个进程
3. 调度程序选择这个进程



扫描全能王 创建

操作系统分类:

大型机OS

服务器OS

多处理OS

个人计算机OS

掌上计算机OS

嵌入式计算机OS

传感节点OS

实时OS

智能卡OS

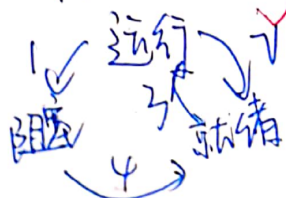
进程的状态

进程

运行态(此时进程占有CPU)

就绪态(等待运行)

阻塞态(除非外部事件发生, 否则进程不能运行)



设计

2) 线程

- 线程是进程中的一个实体，是被系统独立调度和分派的基本单位。
- 线程自己基本上不拥有系统资源，只拥有一点在运行中必不可少的资源(如程序计数器、一组寄存器和栈)，但它可与同属一个进程的其它线程共享进程所拥有的全部资源。

3) 进程线程区别

线程：具有许多传统进程所具有的特征，因此又称“轻型进程” Light-Weight Process

传统进程：又称为“重型进程” Heavy-Weight Process

① i. 调度

- 传统OS中，拥有资源的基本单位和独立调度、分派的基本单位是进程；引入线程的OS中，把线程作为调度、分派的基本单位，而把进程作为资源拥有的基本单位。(即两个属性分开了)
- 在同一进程中，线程的切换不会引起进程的切换；在由一个进程中的线程切换到另一个进程中的线程时会引起进程的切换。

② ii. 并发性

- 引入线程的OS中，不仅进程之间可以并发执行，而且在一个进程中的多个线程之间也可以并发执行，因此更具有并发性，更有效地使用系统资源和提高系统吞吐量。(多线程的文件服务器)

③ iii. 拥有资源

- 线程自己不拥有系统资源，但可以访问其隶属进程的资源，即一个进程的代码段、数据段以及系统资源(已打开的文件、I/O设备)

④ iv. 系统开销

- 进程的开销明显大于线程的开销，无论是创建、撤销、转换。

⑤ v. 从地址空间资源上比较

- 不同的进程的地址空间是相互独立的，而同一进程的各线程共享同一地址空间，一个进程中的线程在另一个进程中是不可见的

⑥ vi. 从通信关系上比较

- 进程间通信必须使用操作系统提供的进程间通信机制，而同一进程的各线程可通过直接读写进程数据段来进程通信，(当然同一进程的各线程间通信也需要同步和互斥手段的辅助，以保证数据一致性)

⑦ vii. 从调度切换上比较

- 同一进程中的线程上下文切换比进程的上下文切换要快得多

11 进程间通信的机制

- 共享内存
- 消息传递
- 管道通信
- 远程过程调用

共享内存

在相互通信的进程之间设有一个公共内存区，一组进程向该公共内存中写，另一组



进程从公共内存中读，通过这种方式实现两组进程间的信息交换。

消息传递系统

进程间的数据交换以消息message为单位

- 计算机网络中，又将message称为报文
- 程序员直接利用系统提供的一组通信原语来实现通信
- Send(destination, &message)
- Receive(source, &message)

管道通信

- 管道通信基于文件系统
- 是指用于连接一个读进程和一个写进程，以实现它们之间通信的共享文件，又称pipe文件。
- 向管道提供输入的发送进程即写进程，以字符流形式将大量的数据送入管道；而接收管道输出的接收进程即读进程，可从管道中接收数据。

远程过程调用

第三方的客户程序通过接口调用系统内部的标准或自定义函数，获得函数返回的数据进行处理后显示或打印。

12. 进程间互斥的手段（怎么实现互斥？）

- i. 屏蔽中断
- ii. 锁变量
- iii. 严格轮换法
- iv. Peterson解法
- v. TSL指令
- vi. Swap指令
- vii. 信号量

屏蔽中断

使每个进程在刚刚进入临界区后立即屏蔽所有中断，并在就要离开之前再打开中断。

缺点：将关闭中断的权利交给用户进程，不明智。

锁变量

设置一个共享变量0与1等，先测试再进入临界区

问题：锁变量仍然是临界资源

严格轮换法

- 严格轮流进入临界区
- 缺点：
 - 忙等待：持续测试一个变量直到某个值出现
 - 运行速度不匹配：进程速度将取决于最慢的进程，会使得整个进程速度变慢
 - 进程0被一个临界区之外的进程阻塞，违反了临界区外运行的进程不得阻塞其它进程。
- 用于忙等待的锁：自旋锁（spin lock）



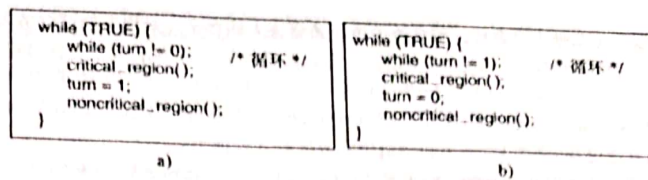


图2-23 临界区问题的一种解法（在两种情况下请注意分号终止了while语句）：a) 进程0，b) 进程1

Peterson解法

```
#define FALSE 0
#define TRUE 1
#define N 2 /*number of processes*/
int turn; /*whose turn is it?*/
int interested[N]; /*all values initially 0*/
void enter_region(int process)
{
    int other;
    other=1-process;
    interested[process]=TRUE;
    turn=process;
    while(turn==process && interested[other]==TRUE);
}
void leave_region(int process)
{
    interested[process]=FALSE;
}
```

TSL指令（硬件）

- (test and set lock) 测试并上锁
- 工作原理：其将内存中一个指定的存储器字读到一个指定的寄存器中，然后在该内存地址上存一个非零值。读数与写数操作是绝对不可分割的。

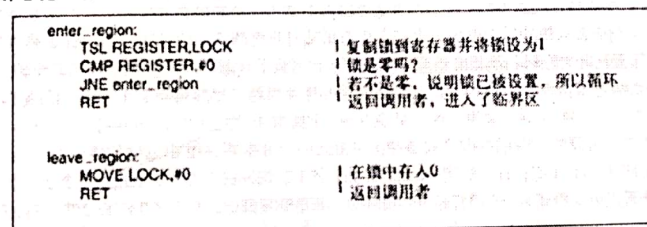


图2-25 用TSL指令进入和离开临界区

Swap指令

- 交换指令：XCHG指令，用于交换两个字的内容，可替代TSL指令

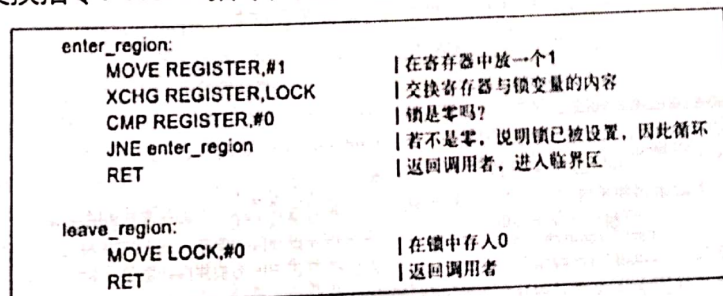


图2-26 用XCHG指令进入和离开临界区

13. 进程间同步的方法（再找找）

i. 信号量

两个或多个进程可以通过简单的信号进行合作，一个进程可以被迫在某一位置



停止,直到它接收到一个特定的信号。该信号即为信号量s。为通过信号量s传送信号,进程可执行原语semSignal(s);为通过信号量s接收信号,进程可执行原语semWait(s);如果相应的信号仍然没有发送,则进程被阻塞,直到发送完为止。

ii. 管程

管程是一个由过程、变量及数据结构等组成的一个集合,它们组成一个特殊的模块或软件包。管程通过使用条件变量提供对同步的支持,这些条件变量包含在管程中,并且只有在管程中才能被访问。

iii. 消息传递

消息传递的实际功能以一对原语的形式提供:

- send(destination,message)
- receive(source,message)

这是进程间消息传递所需要的最小操作集。

一个进程以消息的形式给另一个指定的目标进程发送消息;

进程通过执行receive原语接收消息, receive原语中指明发送消息的源进程和消息。

条件变量、互斥量
加锁、硬件互斥
事件计数器

14. 临界资源、临界区、竞态

- **竞态**: 当类似两个或多个进程读写某些共享数据,而最后的结果取决于进程运行的精确时序的,称为竞争条件。
- **临界资源**: 多进程存在时必须互斥地访问的资源,即使多个进程之间产生竞态的资源。
- **临界区**: 把对临界资源进行访问的程序片段称作临界区域或临界区。

15. 避免竞态的四个条件:

- 任何两个进程不能同时处于临界区
- 不应该对CPU的速度和数目做任何假设
- 临界区外运行的进程不得阻塞其它进程
- 不得使进程在临界区外无休止地等待

16. 引入线程时 原来进程模型的函数会不会有问题 比如全局数据 特权指令 怎么解决? (引入线程后,不可重入库函数的问题,怎么解决?)

• 全局变量 (√)

- way1: 全面禁止全局变量 (会同许多已有软件冲突)
- way2: 映射为线程私有变量
- way3: 引入新的库过程,以便创建、设置和读取这些线程范围的全局变量。

```
create_global("bufptr");  
set_global("bufptr", &buf);  
bufptr = read_global("bufptr");
```

✓ 不可重入库函数的问题

- such as malloc, stdio functions

- way1: 重写库过程

- way2: 为每个过程提供包装器, 同一个过程一个时间只允许一个线程执行

• 信号接收处理问题



- 线程专用信号、进程信号
- 多线程的堆栈管理问题
 - 传统进程中堆可自动增长，而多线程环境下要三思
 - 重写库的同时，还要考虑兼容原有程序的问题

17. 经典的IPC问题

生产者消费者问题

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

/* 缓冲区中的槽数目 */
/* 信号量是一种特殊的整型数据 */
/* 控制对临界区的访问 */
/* 计数缓冲区的空槽数目 */
/* 计数缓冲区的满槽数目 */

/* TRUE 是真 */
/* 产生放在缓冲区中的一些数据 */
/* 将空槽数目减1 */
/* 进入临界区 */
/* 将新数据项放到缓冲区中 */
/* 离开临界区 */
/* 将满槽的数目加1 */

/* 无限循环 */
/* 将满槽数目减1 */
/* 进入临界区 */
/* 从缓冲区中取出数据项 */
/* 离开临界区 */
/* 将空槽数目加1 */
/* 处理数据项 */

图2-28 使用信号量的生产者-消费者问题

哲学家就餐问题

```
#define N 5 //哲学家数量
#define LEFT (i+N-1)%N //i左邻居编号
#define RIGHT (i+1)%N //i右邻居编号
#define THINKING 0 //哲学家在思考
#define HUNGRY 1 //哲学家试图拿起叉子
#define EATING 2 //哲学家进餐
typedef int semaphore;
int state[N]; //数组用来跟踪每位哲学家的状态
semaphore mutex=1; //临界区的互斥
semaphore s[N]; //每个哲学家一个信号量
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks(i);
    }
}

void take_forks(int i){
    down(&mutex); //进入临界区
    state[i]=HUNGRY;
    test(i); //尝试获得2把叉子
    up(&mutex); //离开临界区
    down(&s[i]); //如果得不到需要的叉子则阻塞
}

void put_forks(int i){
    down(&mutex);
    state[i]=THINKING; //哲学家就餐完毕
    test(LEFT); //检查左边的邻居现在可以吃吗
    test(RIGHT); //检查右边的邻居现在可以吃吗
    up(&mutex);
}
```



```

down(&mutex);
state[i]=THINKING; //哲学家就餐完毕
test(LEFT); //检查左边的邻居现在可以吃吗
test(RIGHT); //检查右边的邻居现在可以吃吗
up(&mutex);
}
void test(int i){
    if(state[i]==HUNGRY && state[LEFT]!=EATING
    && state[RIGHT]!=EATING){
        state[i]=EATING;
        up(&s[i]); //一旦这边up(&s[i]), take_forks函数中就可以down(&s[i])了, 可以去吃饭啦~
    }
}
}

```

读者写者问题✓

读者优先

```

int count = 0; // 用于记录当前的读者数量
semaphore mutex = 1; // 用于保护更新count变量时的互斥
semaphore rw = 1; // 用于保证读者和写者互斥地访问文件
// 写者进程
void writer () {
    while(1){
        down(&rw); // 互斥访问共享文件
        writing(); // 写入
        up(&rw); // 释放共享文件
    }
}
// 读者进程
void reader () {
    while(1){
        down(&mutex); // 互斥访问count变量
        if (count == 0) // 当第一个读进程读共享文件时
            down(&rw); // 阻止写进程写
        count++; // 读者计数器加1
        up(&mutex); // 释放互斥变量count
        reading(); // 读取
        down(&mutex); // 互斥访问count变量
        count--; // 读者计数器减1
        if (count == 0) // 当最后一个读进程读完共享文件
            up(&rw); // 允许写进程写
        up(&mutex); // 释放互斥变量 count
    }
}
}

```

写者优先



```

typedef int semaphore
int wcount = 0;
int rcount = 0;
semaphore wsem = 1;
semaphore rsem = 1;
semaphore x = 1; // 1个读count
semaphore y = 1; // 1个写wcount
semaphore z = 1; // 1个写wcount
// 写者进程
void writer () {
    while(1){
        down(&y);
        wcount++;
        if (wcount == 1)
            down(&rsem);
        up(&y);
        down(&wsem);
        writing();
        up(&wsem);
        down(&y);
        wcount--;
        if (wcount == 0)
            up(&rsem);
        up(&y);
    }
}
// 读者进程
void reader () {
    while (1){
        down(&z); // 因为是写者优先，第一个读者被阻塞在rsem，其他的读者
        // 被阻塞在z，这样在哪拍写者没了，如果在第一个读者读的过程中来了写者，下一步会
        // 让写者继续写。
        down(&rsem);
        down(&x);
        rcount++;
        if (rcount == 1)
            down(&wsem);
        up(&x);
        up(&rsem);
        up(&z);
        reading();
        down(&x);
        rcount--;
        if (rcount == 0)
            up(&wsem);
        up(&x);
    }
}
}

```

睡眠的理发师



扫描全能王 创建

```

// #define CHAIRS 3
typedef int semaphore;
semaphore customers = 0;
semaphore barbers = 0; //用于保证理发师空闲
semaphore mutex = 1; //保护waiting
int waiting = 0;
void barber (void) {
    while (1) {
        down(&customers); // 没人理发
        down(&mutex);
        waiting = waiting - 1;
        up(&barbers);
        up(&mutex);
        cut_hair();
    }
}
void customer (void) {
    down(&mutex);
    if (waiting < CHAIRS) {
        waiting = waiting + 1;
        up(&customers); // 理发师
        up(&mutex);
        down(&barbers);
        get_haircut();
    }
    else {
        up(&mutex);
    }
}
}

```

18. 批处理系统中的调度服务 (调度策略) (会算!!)

• 先来先服务 (FCFS)

• 最短作业优先 (SJF)

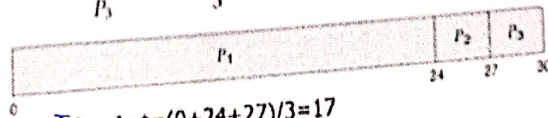
• 最短剩余时间优先 (最短作业优先的抢占式)

◦ 调度程序总是选择剩余运行时间最短的那个进程运行。必须提前掌握运行时间。当一个新的作业到达时，其整个时间同当前进程的剩余时间作比较。如果新的进程比当前运行进程需要更少的时间，当前进程就被挂起，而运行新的进程。

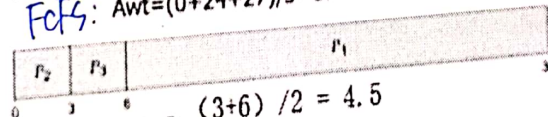
First-come first-served

▪ Average waiting time 平均等待时间

Process	Burst Time
P_1	24
P_2	3
P_3	3



FCFS: $Awt = (0 + 24 + 27) / 3 = 17$



SJF:

$Awt = ?$ $(3 + 6) / 2 = 4.5$

weixudong@nankai.edu.cn



Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

会算!!!

$$SJF: AWT = ? \quad (0+3+9+16) / 4 = 7$$

$$FCFS: AWT = ? \quad (0+6+14+21) / 4 = 10.25$$

- a. SJF: $AWT = (0+3+9+16) / 4 = 7$
 b. FCFS: $AWT = (0+6+14+21) / 4 = 10.25$

▪ i.e. Preemptive SJF scheduling

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

非抢占式

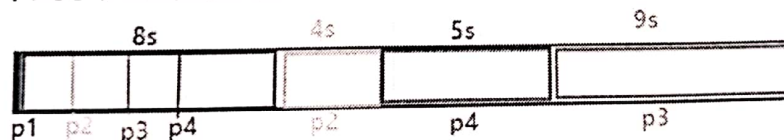
Nonpreemptive SJF scheduling: $AWT = ?$ Preemptive SJF scheduling: $AWT = ?$

抢占式

a. 非抢占式

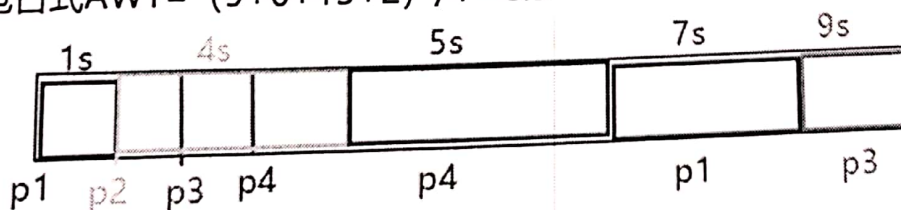
下面的公式分别对应 p_1 、 p_2 、 p_3 、 p_4 的等待时间。

第一个到了先运行起来。



$$AWT = (0+7+15+9) / 4 = 7.75$$

b. 抢占式 $AWT = (9+0+15+2) / 4 = 6.5$



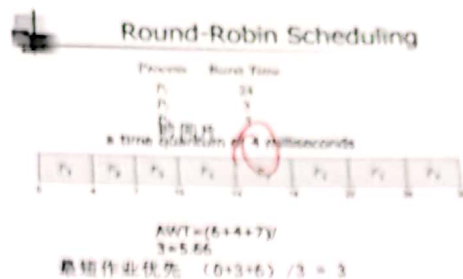
画个时间轴，比较一下到达时间和开始时间就知道等待时间了

19. 交互式系统中的调度 (不知道考哪个调度策略...)

a. 轮转调度

- 每个进程被分配一个时间段，称为时间片，即允许该进程在该时间段中运行。





b. 优先级调度

- 每个进程被赋予一个优先级，允许优先级最高的可运行进程先运行。

c. 多级队列

- 设立优先级类，当一个进程用完分配的时间片后，它被移到下一类。

d. 最短进程优先

- 首先运行最短的作业

e. 保证调度

- 向用户做出明确的性能保证。

f. 彩票调度

- 为进程提供各个系统资源（如CPU时间）的彩票。一旦需要做出项调度决策时，就随机抽取一张彩票，拥有该彩票的进程获得该资源。

g. 公平分享调度

- 无论一个用户有多少进程存在，每个用户都会得到应有的CPU份额。

20. 何为死锁？产生死锁的原因？

死锁：

是指多个进程因竞争资源而造成的一种僵持状态。若无外力作业，这些进程都将永远处于阻塞状态，不能再运行下去。

产生死锁的原因：

- 资源不足
- 进程推进次序不当

21. 资源死锁的四个必要条件

- 互斥条件。**每个资源要么分配给一个进程，要么就是可用的。
- 占有和等待条件。**已经得到了某个资源的进程可以再请求新的资源。
- 不可抢占条件。**已经分配给一个进程的资源不能强制性地被抢占，它只能被占有它的进程显式地释放。
- 环路等待条件。**死锁发生时，系统中一定有两个或两个以上的进程组成一条环路，该环路中的每个进程都在等待着下一个进程所占有的资源。

死锁发生时，以上四个条件一定是同时满足的。如果其中任何一个条件不成立，死锁就不会发生。

22. 四种处理死锁的策略（怎么解决死锁问题）

- 忽略该问题，也许如果你忽略它，它也忽略你。
- 检测死锁并恢复。让死锁发生，检测它们是否发生，一旦发生死锁，采取行动解决问题。

死锁的恢复：
利用抢占恢复
利用回滚恢复
通过杀死进程恢复



- 3) 仔细对资源进行分配, 动态地避免死锁。
 4) 通过破坏引起死锁的四个必要条件之一, 防止死锁的发生。
 21. 安全状态与不安全状态, 不安全状态是死锁吗?
 安全:

已有最大 数量需求		
A	3	9
B	2	4
C	2	7

空闲: 3
a)

已有最大 数量需求		
A	3	9
B	4	4
C	0	7

空闲: 1
b)

已有最大 数量需求		
A	3	9
B	0	-
C	2	7

空闲: 5
c)

已有最大 数量需求		
A	3	9
B	0	-
C	7	7

空闲: 0
d)

已有最大 数量需求		
A	3	9
B	0	-
C	0	-

空闲: 7
e)

图6-9 说明a中的状态为安全状态

不安全:

已有最大 数量需求		
A	3	9
B	2	4
C	2	7

空闲: 3
a)

已有最大 数量需求		
A	4	9
B	2	4
C	2	7

空闲: 2
b)

已有最大 数量需求		
A	4	9
B	4	4
C	2	7

空闲: 0
c)

已有最大 数量需求		
A	4	9
B	-	-
C	2	7

空闲: 4
d)

图6-10 说明b中的状态为不安全状态

★ 注意: **不安全状态并不是死锁。** ✓✓✓

安全状态和不安全状态的区别:

从安全状态出发, 系统能够保证所有进程都可能完成; 而从不安全状态出发, 就没有这样的保证。

22. 死锁避免——银行家算法 (动态预防)

单个资源的银行家算法

算法判断对请求的满足是否会导致进入不安全状态。如果是, 就拒绝请求; 如果满足请求后系统仍然是安全的, 就予以分配。

将客户比作进程, 贷款单位比作资源, 银行家比作操作系统。

已有最大 数量需求		
A	0	6
B	0	5
C	0	4
D	0	7

空闲: 10
a)

已有最大 数量需求		
A	1	6
B	1	5
C	2	4
D	4	7

空闲: 2
b)

已有最大 数量需求		
A	1	6
B	2	5
C	2	4
D	4	7

空闲: 1
c)

图6-11 三种资源分配状态: a) 安全; b) 安全; c) 不安全

银行家算法:

对每一个请求进行检查, 检查如果满足这一请求是否会到达安全状态。若是, 那么就满足该请求; 否则, 就推迟对这一请求的满足。为了检查状态是否安全, 银行家需要考虑他是否有足够的资源满足某一个客户。如果可以, 那么这笔贷款就是能够收回的, 并且接着检查最接近最大限额的一个客户, 以此类推。如果所有投资最终都能被收回, 那么该状态是安全的, 最初的请求可以批准。

多个资源的银行家算法



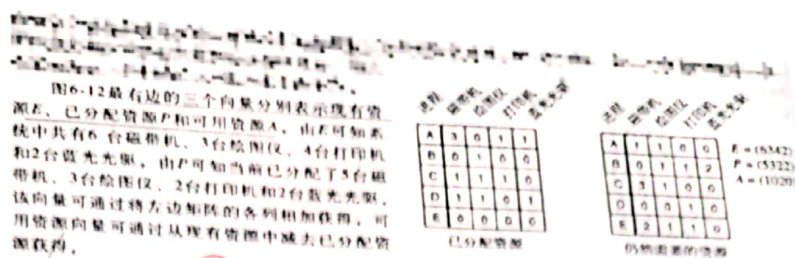


图6-12最右边的三个向量分别表示现有资源E、已分配资源P和可用资源A。由E可知系统中共有6台磁带机、3台绘图仪、4台打印机和2台激光光驱。由P可知当前已分配了5台磁带机、3台绘图仪、2台打印机和2台激光光驱。该向量可通过将右边矩阵的各列相加获得。可用资源向量可通过从现有资源中减去已分配资源获得。

检查一个状态是否有安全的算法如下：

- 1) 查找右边矩阵中是否有一行，其没有被满足的资源数均小于或等于A。如果不存在这样的行，那么系统将会死锁；因为任何进程都无法运行结束（假定进程会一直占有资源直到它们终止为止）。
- 2) 倘若找到这样一行，那么可以假设它获得所需的资源并运行结束，将该进程标记为终止，并将其资源加到向量A上。
- 3) 重复以上两步，或者直到所有的进程都被标记为终止。其初始状态是安全的，或者所有进程的资源需求都得不到满足，此时就是发生死锁。

如果在第1步中同时有若干进程符合条件，那么不管挑选哪一个运行都没有关系，因为可用资源或者会增多，或者至少保持不变。

把已分配的
资源加上
A上:

$A(2,2,0) + P(5,3,2) = A(7,5,2)$
 $A(7,5,2) + P(3,2,2) = A(10,7,4)$
 $A(10,7,4) + P(1,1,1) = A(11,8,5)$

银行家练习：

1. 某系统有R1、R2和R3共3种资源，在T0时刻P1、P2、P3和P4 这4个进程对资源的占用和需求情况见下表，此时系统的可用资源向量为(2, 1, 2)，问题：
- ①将系统中各种资源总数和此刻各进程对资源的请求数利用向量或矩阵表示出来；
- ②如果此时P1和P2均发出资源请求向量Request(1,0,1)，为了保证系统的安全性，应该如何分配资源给这两个进程，说明你所采用策略的原因。
- ③如果②中两个请求立即得到满足后，系统此刻是否处于死锁状态？

(中国科学院软件研究所1999试题)

	最大资源需求量			已分配资源数量		
	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0
P2	6	1	3	4	1	1
P3	3	1	4	2	1	1
P4	4	2	2	0	0	2

① 已分配 $P = (7, 2, 4)$
 可获资源 $A = (2, 1, 2)$
 总: $E = (9, 3, 6)$

解：① 系统中资源总量为某时刻系统中可用资源量与各进程已分配资源量之和，所以各种资源总数为(9,3,6)。

各进程对资源的需求量为各进程对资源的最大需求量与进程已分配资源量之差，即

$$\begin{bmatrix} 3 & 2 & 2 \\ 6 & 1 & 3 \\ 3 & 1 & 4 \\ 4 & 2 & 2 \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 1 \\ 2 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 2 \\ 2 & 0 & 2 \\ 1 & 0 & 3 \\ 4 & 2 & 0 \end{bmatrix}$$

$$\begin{pmatrix} 3 & 2 & 2 \\ 6 & 1 & 3 \\ 3 & 1 & 4 \\ 4 & 2 & 2 \end{pmatrix} - \begin{pmatrix} 1 & 0 & 0 \\ 4 & 1 & 1 \\ 2 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 2 & 2 & 2 \\ 2 & 0 & 2 \\ 1 & 0 & 3 \\ 4 & 2 & 0 \end{pmatrix}$$

② 若此时P1发出资源请求Request₁(1,0,1)，按银行家算法进行检查：

- Request₁(1,0,1) ≤ Need₁(2,2,2)
- Request₁(1,0,1) ≤ Available(2, 1, 2)
- 试分配并修改数据结构，资源分配情况如下：

	Allocation			Need			Available		
P1	2	0	1	1	2	1	1	1	1
P2	4	1	1	2	0	2			
P3	2	1	1	1	0	3			
P4	0	0	2	4	2	0			

- 再利用安全性算法检查系统是否安全，可用资源Available(1,1,1)已不能满足任何进程，故系统进入不安全状态，此时系统不能将资源分配给P1。

若此时P2发出资源请求Request₂(1,0,1)，按银行家算法进行检查：

- Request₂(1,0,1) ≤ Need₂(2,0,2)

$A(1, 1, 1)$

$P1: 100$
 $P2: 512$
 $P3: 211$
 $P4: 002$

222
 101
 103
 420

$P2 \Rightarrow P3 \Rightarrow P4 \Rightarrow P1$
 $A(6, 2, 3) \quad A(8, 3, 4) \quad A(8, 3, 6) \quad A(9, 3, 6)$



- Request: (1, 0, 4) ≤ Available: (2, 1, 2)
- 试分配并修改数据结构, 资源分配情况如下:

	Work				Need				Allocation				Work + Allocation				Finish
P2	1	1	1	1	0	1	5	1	2	6	2	3	6	2	3	6	true
P3	6	2	3	1	0	3	2	1	1	8	3	4	8	3	4	8	true
P4	8	3	4	4	2	0	0	0	2	8	3	6	8	3	6	8	true
P1	8	3	6	2	2	2	1	0	0	9	3	6	9	3	6	9	true

- 从上述分析中可以看出, 此时存在一个安全序列 P2, P3, P4, P1, 故该状态是安全的, 可以立即将 P2 所申请的资源分配给它。

③ 如果②中两个请求立即得到满足后, 系统此刻并没有立即进入死锁状态, 因为这时所有进程没有提出新的资源申请, 全部进程均没有因资源请求没有得到满足而进入阻塞状态, 只有当进程提出资源申请且全部进程都进入阻塞状态时, 系统才处于死锁状态。

23. 死锁预防

- 1) 破坏互斥条件: 一切都使用假脱机技术。
- 2) 破坏占有并等待条件: 在开始就请求全部资源。
- 3) 破坏不可抢占条件: 抢占资源。
- 4) 破坏环路等待条件: 对资源按序编号。

内存管理

24. 分页和分段机制

1) 分段

- ★ 二维作业地址空间: 段号 + 段内偏移量

- ★ 基本原理: 连续分配内存。

将作业的地址空间被划分为若干个段, 每个段定义了一组逻辑信息。每个段都有自己的名字以及“段号”。

每个段都从 0 开始编址, 并采用一段连续的地址空间; 各段间可离散分布于物理内存中

段的长度由相应的逻辑信息组的长度决定, 故各段长度不等

提供一个二维线性虚拟空间

2) 分页

- ★ 一维作业地址空间: 地址

原理: 离散性、多次性、对换性、虚拟性

把程序的逻辑地址空间划分为若干页, 页内地址和物理地址是一一对应的, 因此只需将逻辑地址中的页号转换为内存中的物理块号。

3) 分页与分段的比较

- ★ 相同点

- 都采用离散分配方式
- 都通过地址映射机构来实现地址变换

不同点 (概念与目的都不同)

- 页是信息的物理单位, 分页是为实现离散分配方式, 以消减内存的外零头, 提高内存的利用率; 分页是由于系统管理的需要而不是为用户的需要;
- 段是信息的逻辑单位, 它含有一组有意义相对完整的信息。分段的目的是为了能更好的满足用户的需要;
- 页的大小固定且由系统确定, 并由机器硬件实现的, 因而一个系统只能有

基本内存管理

单一连续分配

固定分区分配

动态分区分配

动态重定位分区分配

交换技术

分页存储管理

分段存储管理

段页式存储管理

虚拟内存管理

特点:

请求分页系统

请求分段系统

连续分配易产生“碎片”

用内存紧凑/压缩

来解决

碎片

逻辑



Request: (1,0,1) ≤ Available: (2,1,2)

试分配并修改数据结构，资源分配情况如下：

	Work			Need			Allocation			Work + Allocation			Finish
P2	1	1	1	1	0	1	5	1	2	6	2	3	true
P3	6	2	3	1	0	2	1	1	8	3	4	true	
P4	8	3	4	4	2	0	0	0	2	8	3	6	true
P1	8	3	6	2	2	2	1	0	0	9	3	6	true

为下-丁的work集

只有当进程提出资源申请且全部进程都进入阻塞状态时，系统才处于死锁状态。

从上述分析中可以看出，此时存在一个安全序列P2, P4, P1, P3，故该状态是安全的，可以立即将P2所申请的资源分配给它。

③ 如果要求两个请求立即得到满足后，系统此刻并没有立即进入死锁状态，因为这时所有进程没有提出新的资源申请，全部进程均没有因资源请求没有得到满足而进入阻塞状态，只有当进程提出资源申请且全部进程都进入阻塞状态时，系统才处于死锁状态。

23. 死锁预防

- 1) 破坏互斥条件：一切都使用假脱机技术。
- 2) 破坏占有并等待条件：在开始就请求全部资源。
- 3) 破坏不可抢占条件：抢占资源。
- 4) 破坏环路等待条件：对资源按序编号。

内存管理

24. 分页和分段机制

1) 分段

★ • 二维作业地址空间：段号 + 段内偏移量

• 基本原理

将作业的地址空间被划分为若干个段，每个段定义了一组逻辑信息。每个段都有自己的名字以及“段号”。

• 每个段都从0开始编址，并采用一段连续的地址空间；各段间可离散分布于物理内存中

• 段的长度由相应的逻辑信息组的长度决定，故各段长度不等

• 提供一个二维线性虚拟空间

2) 分页

★ • 一维作业地址空间：地址

原理：离散性、多发性、对换性、虚拟性

• 把程序的逻辑地址空间划分为若干页，页内地址和物理地址是一一对应的，因此只需将逻辑地址中的页号转换为内存中的物理块号。

3) 分页与分段的比较

• 相同点

- 都采用离散分配方式
- 都通过地址映射机构来实现地址变换

• 不同点 (概念与目的都不同)

- 页是信息的物理单位，分页是为实现离散分配方式，以消减内存的外零头，提高内存的利用率；分页是由于系统管理的需要而不是为用户的需要；
- 段是信息的逻辑单位，它含有一组有意义相对完整的信息。分段的目的是为了能更好的满足用户的需要；
- 页的大小固定且由系统确定，并由机器硬件实现的，因而一个系统只能有

基本内存管理

单一连续分配

固定分区分配

动态分区分配

动态重定位分区分配

交换技术

分页存储管理

分段存储管理

段页式存储管理

虚拟内存管理 → 特点

请求分页系统

请求分段系统

连续分配易产生“碎片”

用内存紧张/压缩

快解决

部碎片

逻辑



一种大小的页面；而段的长度却不固定，且其由用户所编写的程序(或编译程序)根据信息的性质来划分；

- 分页的作业地址空间是一维的，即单一的线性地址空间，程序员只须利用一个记忆符，即可表示一个地址；分段的作业地址空间是二维的，程序员在标识一个地址时，既需给出段名，又需给出段内地址。

25. 局部性原理

- 时间局部性：若程序中的某条指令一旦执行，则不久以后该指令可能再次执行；如果某个数据结构被访问，则不久以后该数据结构可能再次被访问；
- 空间局部性：一旦程序访问了某个存储单元，在不久以后，其附近的存储单元也被访问。

26. 虚拟内存（不一定考）

概念：

- 使程序在只有一部分被调入内存的情况下运行。
- 也即具有请求调入功能和置换功能，能从逻辑上对内存容量进行扩充的一种存储器系统

实现：

其是建立在离散分配存储管理方式的基础上的。

- 分页请求系统
- 请求分段系统

27. 页面置换算法（会算！）

- 最优页面置换算法
- FIFO先进先出页面置换算法
- LRU最近最久未使用页面置换算法
- 第二次机会页面置换算法/时钟Clock置换算法
- 页面缓冲算法
- 工作集页面置换算法/工作集时钟页面置换算法wsclock

最优页面置换算法

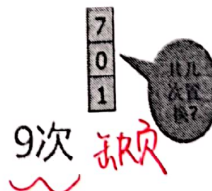
- 其所选择的淘汰页面，将是永不使用的，或者是在最长时间不再访问的页面。
- 缺点：无法预知那个是未来最长时间不再访问的。

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
页面号	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
引用串																				



页框 (物理块)	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
	7	0	0	0	0	4	3	3	3	3	3	3	3	3	3	3	3	3	3	3

最佳页面置换算法的置换图



1次置换

FIFO先进先出页面置换算法

- 该算法总是淘汰最先进入内存的页面，即选择在内存中驻留时间最久的页面予以淘汰。



- 缺点：性能较差，置换所依据的条件依赖于页面调入的先后。

★

页面号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
引用串	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	2	0	1
页框物理块	7	7	7	2		2	2	4	4	4	0			0	0			7	7	7
		0	0	0		1	1	1	2	2	2			1	1			1	0	0
			1	1		1	0	0	0	3	3			3	2			2	2	1

FIFO置换算法的置换图

15次

栈内最晚调入的页面

5次读
4次写

第二次机会页面置换算法

- 置换算法在选择一页淘汰时，只须检查其访问位（R位）：如果是0，就选择该页换出；若为1，则重新置它为0，暂不换出而给该页第二次驻留内存的机会，再按照FIFO算法检查下一个页面。
- 近似算法：
 - NRU最近未使用页面置换算法
 - 两位（read, modify）：(1,1)代表最近既被读过也被写过，优先级最高
 - 优先级（高到低）：(1,1) (1,0) (0,1) (0,0)

时钟clock置换算法

（第二次机会页面算法的环形链表实现）

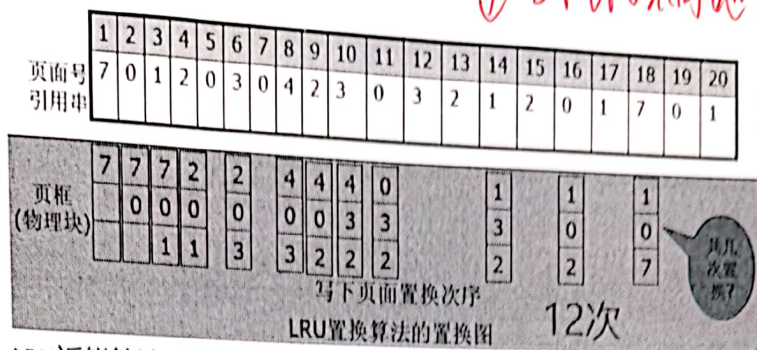
- 该算法只须为每页设置一位访问位，再将内存中的所有页面都通过连接指针连成一个循环队列；当某页被访问时，其访问位被置1；
- 置换算法在选择一页淘汰时，只须检查其访问位（R位）：如果是0，就选择该页换出；若为1，则重新置它为0，暂不换出而给该页第二次驻留内存的机会，再按照FIFO算法检查下一个页面。
- 当检查到队列中的最后一个页面时，若其访问位仍为1，则再返回到队首再去检查第一个页面；
- 该算法会循环地检查各页面的使用情况。

LRU最近最久未使用置换算法

- 根据页面调入内存后的使用情况，由于无法预测各页面将来的使用情况，只能用“最近的过去”作为“最近的将来的近似”。
- 该算法选择最近最久未使用的页面予以淘汰。
- 该算法赋予每个页面一个访问字段，用来记录一个页面自上次被访问以来所经历的时间t，当须淘汰一个页面时，选择现有页面中其t值最大的，即最近最久未使用的页面予以淘汰。
- 实现
 - 硬件：计数器/矩阵
 - 栈方法（栈顶——最新，栈底——最近最久未使用）



扫描全能王 创建



• LRU近似算法 (软件模拟LRU)

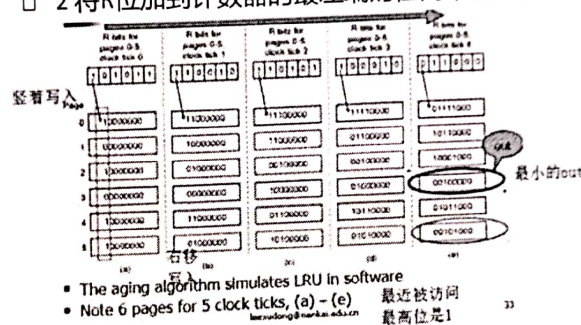
◦ NFU最不频繁使用算法

- 将每个页面与一个软件计数器相关联, 计数器的初值为0。每次时钟中断时, 由操作系统扫描内存中所有的页面, 将每个页面的R位 (值为0/1) 加到它的计数器上。记录了各个页面被访问的频繁程度。发生缺页中断时, 则置换计数器值最小的页面。

◦ 老化算法

▪ 对NFU进行修改

- ◻ 1 在R位被加进之前先将计数器右移一位
- ◻ 2 将R位加到计数器的最左端的位而不是最右端的位。



◦ LFU最少使用置换算法

- 为在内存中的每个页面设置一移位寄存器, 用来记录该页面被访问的频率, 每次访问某页时便对该页移位寄存器的最高位置1, 再每隔一定时间(100ms)右移一次
- 最近一段时间使用最少的页面将是 $R_0 + R_1 + \dots + R_{n-1}$ 最小的页

页面缓冲算法

- 采用可变分配和局部置换方式, 置换算法采用FIFO
- 该算法规定一个被淘汰的页放入两个链表中的一个
 - 如果页面未被修改, 就将它直接放入空闲链表表中;
 - 否则, 便放入已修改页面的链表中。
- 空闲页面链表, 是一个空闲物理块链表, 当需要读入一个页面时, 便可利用空闲页面链表中的第一个物理块, 来装入该页。
- 利用这种方式, 可使已被修改的页面和未被修改的页面都仍留在内存中, 从而显著减少了磁盘I/O的操作次数。

工作集work set页面置换算法

- 工作集: 在某段时间间隔a内, 进程实际要访问的页面的集合。
- 工作集模型: 跟踪进程的工作集 (工作集是最近k次内存访问所访问页面的集)



合，函数 $w(k,t)$ 是在 t 时刻工作集的大小），以确保让程序运行以前，它的工作集就已经在内存中了。

- 目的：大大减少缺页中断率。
- 算法：

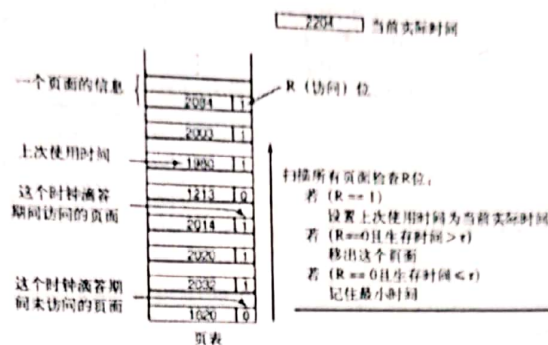


图3-19 工作集算法

工作集时钟页面置换算法wsclock

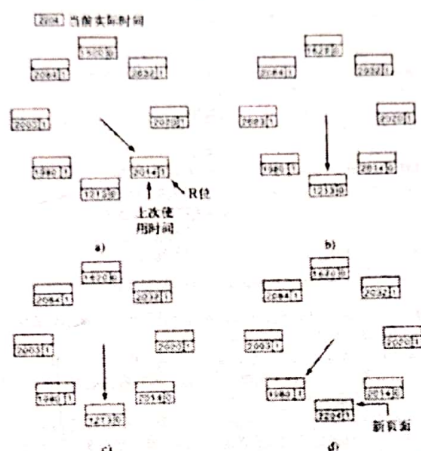


图3-20 工作集时钟页面置换算法的操作：a)和b)给出在 $R=1$ 时所发生的情形；c)和d)给出 $R=0$ 的例子。

练习 (书后题)

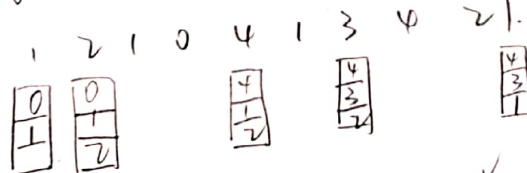
1. 在一个采用页式虚拟存储管理的系统中，某进程依次要访问的字地址是：115, 228, 128, 88, 446, 102, 321, 432, 260, 167。若作业的第0页已经装入内存，现分配给该作业的主存共300字，页面的大小为100字。请回答以下问题：

- (1) 按 FIFO 调度算法将产生多少次缺页中断，依次淘汰的页号是什么？给出计算过程。
- (2) 按 LRU 调度算法将产生多少次缺页中断，依次淘汰的页号是什么？给出计算过程。

依次访问：

1 2 1 0 4 1 3 4 2 1

(1) FIFO



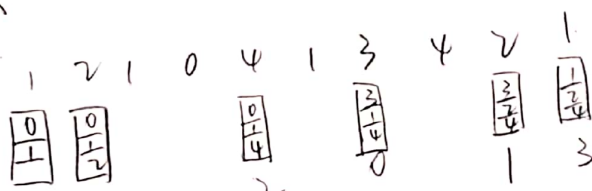
淘汰

缺页缺页
0 1

共5次缺页

依次淘汰：0, 1, 2

(2) LRU



淘汰

共6次缺页

依次淘汰：2, 0, 1, 3



1.

因为页面的大小是 100 字，所以字地址的低四位是页内地址其余部分是页号，因此，进程要依次访问的页面号是：1, 2, 1, 0, 4, 1, 3, 4, 2, 1。
又因为分配给用户的内存共 300 字，即是 $300/100=3$ 帧，(2 分)

(1) FIFO 调度算法执行过程：

1	2	1	0	4	1	3	4	2	1
0	0	0	0	0	4	4	4	4	4
	1	1	1	1	1	1	3	3	3
		2	2	2	2	2	2	2	1

因此，缺页 5 次，依次淘汰的页号是：0, 1, 2。

(2) LRU 调度算法执行过程：

1	2	1	0	4	1	3	4	2	1
0	0	0	0	0	0	0	3	3	1
	1	1	1	1	1	1	1	2	2
		2	2	2	4	4	4	4	4

因此，缺页 6 次，依次淘汰的页号是：2, 0, 1, 3。

2.

因为 $140+80+100+60+50+30+15+20=495\text{KB}<1024\text{KB}$ ，因此，3 个作业都可以分配到所需要内存，此

28. Belady 异常

(自己算一下)

在分页式虚
拟存储器管理中
发生缺页时的
置换算法是 FIFO
算法时，如果对一
进程未分配它所
要求的全部页面，
有时就会出现分面
的页面数增多
但缺页率反而
增高的现象

All pages frames initially empty

	0	1	2	3	0	1	4	0	1	2	3	4
Youngest page		0	1	2	3	0	1	4	4	4	2	3
Oldest page												

P P P P P P P P P P P P

(a)

FIFO with 3 page frames

	0	1	2	3	0	1	4	0	1	2	3	4
Youngest page		0	1	2	3	3	3	4	0	1	2	3
Oldest page												

P P P P P P P P P P P P

(b)

When FIFO with 4 page frames

栈式算法

具有下述性质的算法，称为栈式算法。FIFO 算法没有此性质。

$$M(m, r) \subseteq M(m+1, r)$$

29. 抖动的原因及解决办法

1) 抖动

在虚拟内存中，页面在内存与外存之间频繁调度，以至于调度页面所需时间比进程实际运行时间还多，此时系统效率急剧下降，甚至导致系统崩溃。这种现象称为颠簸

FIFO
3 帧
0 1 2 3 0 1 4 0 1 2 3 4
0 0 0 3 3 3 4 4 4 4 4 4
1 1 1 1 1 1 1 1 1 1 1 1

9 次缺页中断

4 帧
0 1 2 3 0 1 4 0 1 2 3 4
0 0 0 0 4 4 4 4 4 4 4 4
1 1 1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3 3 3

10 次缺页



★ 2) 产生抖动的原因

答案的:

- 页面淘汰算法不合理
- 分配给进程的物理页面数太少
- 抖动原因1: 在请求分页系统中, 通常每一个进程都只能分配到它所需要的全部内存空间中的一部分; 如果进程A在运行过程中需要增加页面, 它便会产生中断。在采用全局置换策略时, 实际上它有可能是被分配到一个原属B进程的物理块, 用来装入新调入的页。而B进程在运行中还需要该物理块, 因而也会产生缺页中断, 于是有可能又获得了C进程的一个物理块, 等等。而这些产生缺页中断的进程, 可能会因页面的调入/调出而处于等待状态, 从而导致就绪队列空。
- 抖动原因2: 通过对CPU的监督, 一旦调度程序发现CPU的利用率降低时, 应立即提高多道程序度, 以提高CPU的利用率。为此, 需引入新的进程参加运行。当新进程进入内存时, 客观上只能从其它运行进程处去获得物理块, 于是又将进一步加剧了进程的缺页情况, 使等待页面调入/调出的进程数目也随之增多, 这又将使CPU的利用率进一步下降。又去引入新的进程, 形成恶性循环。
- CPU的利用率与多道程序度之间的关系, 先增后减。
- 为了保证系统有较大的吞吐量, 必须防止发生抖动

★ 3) 产生抖动的预防

i. 采取局部置换策略

- 把抖动影响局限在单个进程内
- 系统采取可变分配方式, 且采取局部置换策略
- 当某进程发现缺页后, 仅在自己的内存空间范围内置换页面, 不允许从其它进程获得新的物理块。这样即使有某个进程发生抖动, 也不至于引起其它进程也产生抖动

ii. 在CPU调度程序中引入工作集算法

- 调度前检查每个进程在内存中驻留页面是否足够多, 如果够则调入新的作业, 否则为缺页率高的进程增加物理块。

iii. 利用 $L=S$ 准则调节缺页率

- 其用来调整多道程序度, L 是缺页之间的平均时间, S 是处理一次缺页的时间。此时CPU的利用率最好。

iv. 挂起若干进程

- 当多道程序度偏高时, 为了防止发生“抖动”, 挂起一些进程, 以便腾出内存空间来分配给抖动的进程。 *降低多道程序度。*
- 被挂起的进程通常是选择优先权最低或较低的;
- 当内存非常拥挤时, 也可以选择一个并不很重要的, 但确较大的进程挂起, 以便能一次释放出较大的内存空间;
- 或将具有最多剩余执行时间的进程挂起。

29. 怎样提高内存利用率 (再找找, 不一定对)



- 负载控制
- 指令和数据分开存储
- 共享内存
- 共享库
- 清除策略, 预留空闲物理帧
- 虚拟存储接口
- ...

30. CPU二级缓存、一级缓存

- 缓存的工作原理是当CPU要读取一个数据时, 首先从缓存中查找, 如果找到就立即读取并送给CPU处理; 如果没有找到, 就用相对慢的速度从内存中读取并送给CPU处理, 同时把这个数据所在的数据块调入缓存中, 可以使得以后对整块数据的读取都从缓存中进行, 不必再调用内存。
- 使CPU读取缓存的命中率非常高, 节省了CPU直接读取内存的时间, 也使CPU读取数据时基本无需等待。总的来说, CPU读取数据的顺序是先缓存后内存。
- 按照数据读取顺序和与CPU结合的紧密程度, CPU缓存可以分为一级缓存, 二级缓存。每一级缓存中所储存的全部数据都是下一级缓存的一部分。
- CPU要读取一个数据时, 首先从一级缓存中查找, 如果没有找到再从二级缓存中查找, 如果还是没有就从三级缓存或内存中查找。一般来说, 每级缓存的命中率大概都在80%左右, 也就是说全部数据量的80%都可以在一级缓存中找到, 只剩下20%的总数据量才需要从二级缓存、三级缓存或内存中读取, 由此可见一级缓存是整个CPU缓存架构中最为重要的部分。

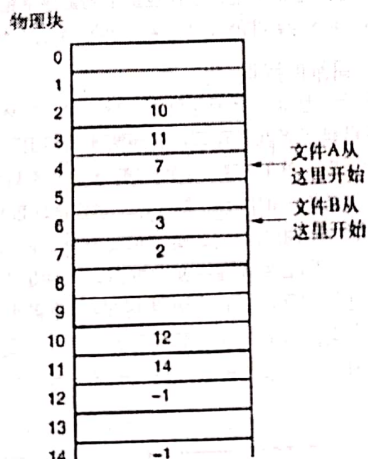
文件系统

31. 文件实现

- 连续分配
- 链表分配
- 采用内存中的表进行链表分配 (FAT)
- i节点

32. FAT表结构

- 取出每个磁盘块的指针字, 把它们放在内存的一个表中。每个磁盘块内存储指向下一个磁盘块的指针。内存中这样的一个表格称为文件分配表 (FAT)。
- 优点: 整个块都可以存放数据; 随机访问也容易很多。
- 缺点: 必须要把整个表都存放在内存中。



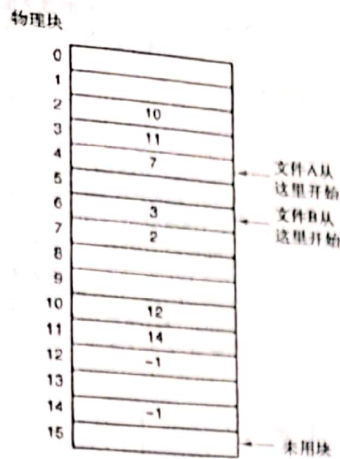
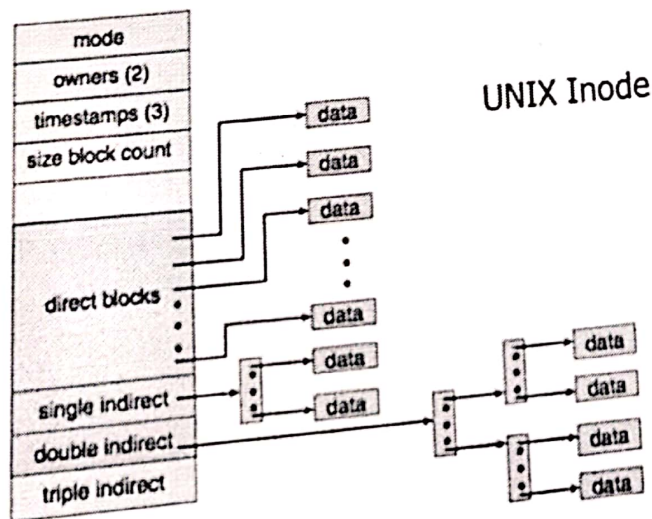


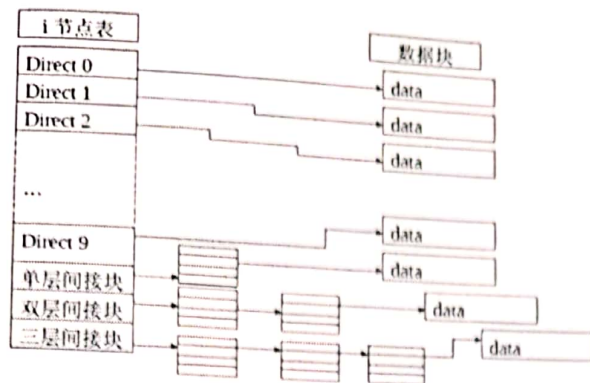
图4-12 在内存中使用文件分配表的链表分配

- 例，文件A依次使用了磁盘块4、7、2、10和12。
33. i节点 (要会算)
- 给每个文件赋予一个称为i节点的数据结构，其中列出了文件属性和文件块的磁盘地址。
 - UNIX索引文件

在i节点表里设下了13个登记项，这些登记项有10个指向**直接块**，一个指向“**一层间接块**”，一个指向“**二层间接块**”，最后一个指向“**三层间接块**”。

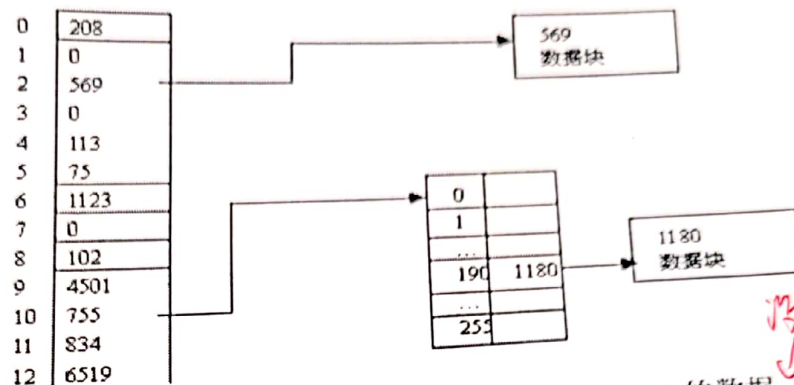
 - 在i节点表中记录数据块地址的就是i节点结构中的i_addr[13]，它的类型是整数，绝大多数UNIX系统中，**整数占4个字节** ($2^2 * 8b = 32b$)。
 - **直接块**：10个指针，这项这个文件前10个数据块（假设每个块512字节）。
 - **一级间接块**：可以存储 $512/4 = 128$ 个地址。直接+一级可以存储小于138个数据块的文件。
 - **二级间接块**：可以存储 2^{14} 个数据块的地址。
 - **三级间接块**： $2^7 * 2^7 * 2^7 = 2^{21}$ 的数据块的地址。





★ • i节点例子 (会算!)

举例:



没物理数据块
107p1

例 1: 某进程要读取某文件字节偏移量 byte offset 为 2488 的数据, 此时核心程序计算出这个字节在第 2 个直接块, 于是它会读取数据块 569, 并在该块第 440 个字节找到所需的信息。440+2*1024=2488。

例 2: 某进程要存取文件 A 内字节偏移量为 204803 的数据? (物理数据块索引值为 4 个字节)

- 1 当物理数据块大小为 1024 时, 如何存取?
- 2 当物理数据块大小为 2048 时, 如何存取?

1. 当物理块大小为 1024 时, 需 $204803/1024=200.0029=201$ 个块存储 (200 个完整数据块+3 字节)。
计算得到, 直接块个数: 10; 一级间接块: $1024/4=256$ 个。因此, 应将文件 A 的数据存储从直接块直至一级间接块的第 191 个块内的第 3 个字节。
2. 当物理块大小为 2048 时, $204803/2048=100.0014=101$ 个块存储 (101 个完整数据块+3 字节)。
计算得到, 直接块个数: 10; 一级间接块个数: $2048/4=512$ 个。因此, 应将文件 A 的数据存储从直接块直至一级间接块的第 91 个块内的第 3 个字节。



存放在某个磁盘上的文件系统，采用混合索引分配方式，其FCB中共有13个地址项，第0—9个地址项为直接地址，第10个地址项为一次间接地址，第11个地址项为二次间接地址，第12个地址项为三次间接地址。如果每个盘块的大小为512字节，则每个索引块可记录170个盘块地址：

(1)该文件系统允许文件的最大长度是多少

(2)长度为80K的文件，需要占用多少个盘块；长度为100K的文件，需要占用多少个盘块；长度为200K的文件，需要占用多少个盘块

(3)假设某个文件的FCB已在内存，但其他信息均在外存，为了访问该文件中某个位置的内容，最少需要几次访问磁盘，最多需要几次访问磁盘

该文件系统的最大长度是 $(10 + 170 + 170 \times 170 + 170 \times 170 \times 170) \times 512B$ ，

(2)长度为80K的文件，需占用文件块， $80K/512 = 160$ 个，则只用到一级索引，因此是160个文件块加上1个索引块，共161个盘块。

长度为100K的文件，需占用文件块 $100K/512 = 200$ 个，则需用到二级索引，在二级索引中用到了第一层索引块1个，第二层索引块1个，因此200个文件块，1个一级索引块，2个二级索引块，共203个盘块。

长度为200K的文件，需占用文件块 $100K/512 = 400$ 个，则需用到二级索引，在二级索引中用到了第一层索引块1个，第二层索引块2个，因此400个文件块，1个一级索引块，3个二级索引块，共404个盘块。

(3)访问该文件中某个位置的内容，最少需要1次访盘，即此位置在直接地址中可以找到，最多需要4次访盘，即在三级索引中找到。

34. 空闲块管理方式

1) 空闲块表

2) 位图

- o n个块的磁盘需要n位位图。
- o 1：空闲，0：已分配。

3) 空闲块链法

- o 把文件存储设备上的所有空闲块链接在一起，从链头分配空闲块，把回收的空闲块插入到链尾

★ 4) 成组连接法 (知道原理！)

o 初始化分组

- 首先把文件存储设备中的所有空闲块按 $T(=50)$ 块划分为一组，组的划分从后往前进行的。其中每组的第一块用来存放前一组中各块的块号和总块数。第一组为 $T-1$ 块。最后一组的物理块号(若干个 $\leq T$)与总块数只能放在管理文件存储设备用的文件资源表中

o 分配过程 (回收过程反之即可)

- 首先，系统在初始化时把文件资源表复制到内存，从而得到最后一组空闲块的物理块号与总块数，并可使得空闲块的分配与释放可以在内存中进行。用于空闲块分配与回收的堆栈的栈指针Ptr(其初值等于该组空闲块总块数)
- 当申请者申请n块空闲块时，按后进先出的原则，分配程序在取走Ptr所指的块号之后，再做 $Ptr = Ptr - 1$ 。这个过程一直持续到所要求的n块空间都分配完毕或堆栈中只剩下最后一个空闲块的块号时
- 当堆栈中只剩下最后一个空闲块号时，系统启动设备管理程序，将该块中存放的下一组的块号与总块数读入内存之后再把该块分配给申请者，然后系统重新设置Ptr指针并继续为申请者分配空间。文件存储设备的最后一个空闲块中设置有尾标识，以指示空闲块分配完毕



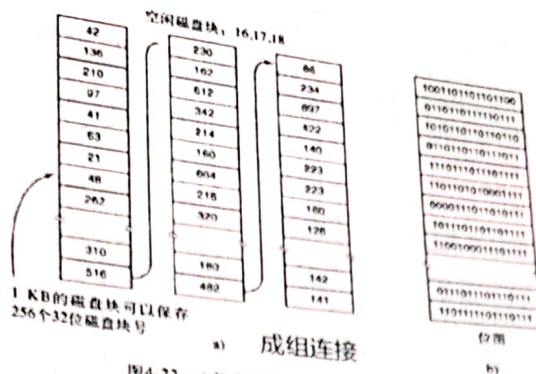


图4-22 a) 把空闲表存放在链表中; b) 位图

35. 文件系统性能优化方法

(可能考吧, 记下小标题)

1) 高速缓存

- 最常用的减少磁盘访问次数的技术是块高速缓存或者缓冲区高速缓存。
- 高速缓存置换算法: FIFO算法、第二次机会算法、LRU算法等, 都适用于高速缓存。
- 与分页相比, 高速缓存的好处在于对高速缓存的引用不很频繁, 所以按精确的LRU顺序在链表中记录全部的块时可行的。

2) 块提前读

- 在用到块之前, 试图提前将其写入高速缓存, 从而提高命中率。
- 局部性原理

3) 减少磁盘臂运动

- 把可能顺序访问的块放在一起, 最好在同一个柱面上, 从而减少磁盘臂的移动次数。

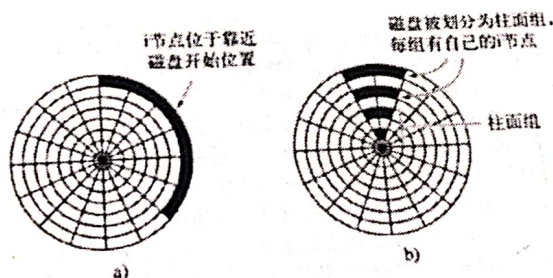


图4-29 a) i节点放在磁盘开始位置; b) 磁盘分为柱面组, 每组有自己的块和节点

- a) 全部i节点都放在靠近磁盘开始位置, 需要较长的寻道时间。
- b) 将磁盘分为多个柱面组, 每个柱面组有自己的i节点, 数据块和空闲表。在文件创建的时, 可任选一i节点, 但选定之后, 首先在该i节点所在的柱面组上查找块。如果在该柱面组中没有空闲块, 就选用与之相邻的柱面组的一个块。

36. 共享文件实现 (好像提了...)

• 符号链接

- 若目录B想共享目录C中的一个文件F, 可以由系统在目录B下创建一个LINK类型的新文件T, 在新文件T中只包含被链接文件F的路径名, 称这样的链接方式为符号链接(Symbolic Linking)。新文件T中的路径名则只被看做是符号链接(Symbolic Link)。



- 在链接的文件F时，可以读LINK类型的链接新文件T，OS将根据新文件T中的路径名去读文件F，于是实现了文件共享。
- 在以符号链接实现文件共享时，只有文件所有者才拥有指向其索引结点的指针；而共享该文件的其他用户（或链接文件）只有该文件的路径名，而不是指向索引结点的指针。
- 当文件的所有者把一个共享文件删除后，其他用户视图通过符号链访问一个被删除的共享文件时，会因系统找不到该文件而使访问失败，于是将符号链删除掉，此时不会发生任何问题。
- 优点：它能够用于链接（通过计算机网络）世界上任何地方的机器中的任何文件的文件。
- 缺点：通过符号链访问共享文件时每次访问的开销大（因为间接访问）。
- 硬链接：不能跨分区，i节点号相同的是一个文件。
 - 缺点：一个目录试图删除文件后，会造成别的目录指向一个错误或者无效节点。
 - 解决办法：只删除C的目录项，i节点保留下来。计数为0时，再删除该文件。

IO设备管理

37. I/O操作的4种方式及其特点（使用的场合？）

1) 程序直接控制

- 这是I/O最简单的方式，又称“忙-等”方式。输入和输出完全是通过CPU执行程序来完成的。
- 该方式执行一个循环程序，反复查询外设状态，如果外设“忙碌”则循环查询直到查得外设状态为“闲置”时止。
- 该方式适用于机内没有中断机构的场合。

2) 中断控制

- 使用中断，允许CPU在等待进程变为就绪的同时做某些其他事情。
- 该方式在进行I/O时，CPU向设备控制器发出I/O命令后便转其他任务的处理，外设操作由设备控制器控制，CPU于外设并行工作。当外设完成I/O后向CPU发中断信号，CPU只需花费很少的时间进行I/O的善后处理，此前无需进行干预。
- 该方式适用于低速设备I/O，并可配合DMA和通道方式实现I/O。

3) DMA控制

- 由DMA控制器而不是主CPU做全部的工作。这一策略需要特殊的硬件（DMA控制器），但是使CPU获得自由从而可以在I/O期间做其他工作。
- 该方式适用于高速外设I/O，一次可以在外设与内存之间传输一个或多个数据块，传输完毕后才需CPU干预。

4) 通道控制

- 通道通过执行通道程序，并与设备控制器来共同实现对I/O设备的控制。
- 该方式中系统预先要将I/O的过程实现为一段通道程序，置于内存的特定位置，而后启动通道。由通道负责执行通道程序对外设进行I/O控制，CPU转其他程序运行。I/O完成后通道向CPU发出中断信号，CPU花很少时间作善后处理。



通道与DMA的比较

- 通道控制方式与DMA方式的比较
 - 相似处
 - 都是以内存为中心，实现设备和内存直接交换数据的控制方式
 - 不同处
 - 在DMA方式中，数据的传送方向、存放数据的内存始址以及传送数据块长度等由CPU控制；
 - 在通道方式下，上述操作都由专管输入、输出的硬件-通道来进行控制
 - 通道控制方式可以做到一个通道控制多台设备与内存进行数据交换，这与DMA方式时每台设备至少有一个DMA控制器不同。

38. 4种控制方式的优缺点 (结合起来答一下)

1) 程序直接控制方式

○ 缺点:

- CPU和IO设备之间只能串行工作
- CPU在同一段时间内只能和一台外设交互，无法实现设备间的并行工作
- 无法发现由设备和其他硬件所产生的错误

2) 中断控制方式

○ 优点

- 解决了主机和外设并行工作的问题；尤其是慢的外设
- 能充分发挥处理器的使用效率；CPU不用无限查询等待
- 提高系统的实时能力

○ 缺点

- 中断驱动I/O是以字节为单位进行I/O的，每完成一个字节的I/O，控制器便要向CPU请求一次中断。若从磁盘读出1KB的数据块，需要中断1K次CPU

3) DMA控制方式

○ 优点

- 数据传输的基本单位是数据块，即CPU与I/O设备之间，每次传送至少是一个数据块
- 所传送的数据是从设备直接送入内存的，或者相反；不需CPU
- 仅在传送一个或多个数据块的开始或结束时，才需CPU干预，整块数据的传送是在DMA控制器的控制下完成的。

○ 缺点

- DMA方式对外围设备的管理和某些操作仍由CPU控制，当外设种类数量较多时，控制也很复杂；
- 多个DMA控制器的同时使用会引起内存地址的冲突，并使控制过程进一步复杂化。

4) I/O通道控制方式

○ 优点

- 在DMA方式中，数据的传送方向、存放数据的内存始址以及传送数据块长度等由CPU控制；在通道方式下，上述操作都由专管输入、输出的硬



- 通道控制方式可以做到一个通道控制多台设备与内存进行数据交换, 这与 DMA 方式时每台设备至少有一个 DMA 控制器不同。
- 39. 磁盘臂调度算法 (会算!)
- 记得指定起始位置
- 先来先服务 FCFS
- 最短寻道时间优先 SSTF
 - 以申请者要求磁头移动距离的大小作为优先的因素
- 电梯算法
 - 不仅仅考虑申请者要求磁头移动方向, 又考虑要求磁头移动的距离, 而且首先方向一致, 其次才是距离最短。
 - 要避免饥饿现象

★ 计算:

记得指定初始位置! 每个人的答案可能都不一样~

设磁盘顺序序列: 11, 1, 36, 16, 34, 9, 12 分别计算寻道距离?

a. FCFS 算法

$$10 + 35 + 20 + 18 + 25 + 3 = 111 \text{ 个柱面}$$

b. SSF 算法

寻道序列: 11, 12, 9, 16, 1, 34, 36

$$\text{距离: } 1 + 3 + 7 + 15 + 33 + 2 = 61 \text{ 个柱面}$$

c. 电梯算法

寻道序列: 11, 12, 16, 34, 36, 9, 1

$$\text{距离: } 1 + 4 + 18 + 2 + 27 + 8 = 60 \text{ 个柱面}$$

练习 (书后题):

1. 假设磁盘有 200 个磁道, 磁盘请求队列中是一些随机请求, 它们按照到达的次序分别处于 98, 183, 37, 122, 14, 124, 65, 67 号磁道上, 当前磁头在 53 号磁道上, 并向磁道号减小的方向上移动。请给出按 FCFS, SSTF, SCAN 及 CSCAN 算法进行磁盘调度时满足请求的次序, 并计算出它们的平均寻道长度。

2. 某个文件系统, 采用混合索引分配方式, 其 FCB 中共有 13 个地址项, 每个盘块的大小为 512 字节, 请回答下列问题:

(1) 如果每个盘块号只需要用 2 个字节来描述, 则该系统需要设置几次间址项?

(2) 如果每个盘块号需要用 3 个字节来描述, 并允许每个盘块中存放 170 个盘块地址, 而且系统采用 10 个直接地址项、1 个一次间址项、1 个二次间址项和 1 个三次间址项, 则对某个长度为 18000000B 的文件, 它需占用多少个盘块 (包括间址块)?

FCFS

下-磁道号	移动磁道数
98	45
183	85
37	146
122	85
14	108
124	110
65	59
67	2

20 + 231 + 218 + 61 = 361 + 279 = 640

SSTF

下-磁道号	移动磁道数
65	12
67	2
37	30
14	23
98	84
122	24
124	2
183	59

14 + 30 = 236 + 8 = 244

SCAN

下-磁道号	移动磁道数
12	41
23	11
30	7
37	7
45	8
53	8
61	8
67	4
75	8
83	8
91	8
98	5
106	8
114	8
122	8
130	8
138	8
146	8
154	8
162	8
170	8
178	8
186	8
194	8
202	8
210	8
218	8
226	8
234	8
242	8
250	8
258	8
266	8
274	8
282	8
290	8
298	8
306	8
314	8
322	8
330	8
338	8
346	8
354	8
362	8
370	8
378	8
386	8
394	8
402	8
410	8
418	8
426	8
434	8
442	8
450	8
458	8
466	8
474	8
482	8
490	8
498	8
506	8
514	8
522	8
530	8
538	8
546	8
554	8
562	8
570	8
578	8
586	8
594	8
602	8
610	8
618	8
626	8
634	8
642	8
650	8
658	8
666	8
674	8
682	8
690	8
698	8
706	8
714	8
722	8
730	8
738	8
746	8
754	8
762	8
770	8
778	8
786	8
794	8
802	8
810	8
818	8
826	8
834	8
842	8
850	8
858	8
866	8
874	8
882	8
890	8
898	8
906	8
914	8
922	8
930	8
938	8
946	8
954	8
962	8
970	8
978	8
986	8
994	8
1002	8
1010	8
1018	8
1026	8
1034	8
1042	8
1050	8
1058	8
1066	8
1074	8
1082	8
1090	8
1098	8
1106	8
1114	8
1122	8
1130	8
1138	8
1146	8
1154	8
1162	8
1170	8
1178	8
1186	8
1194	8
1202	8
1210	8
1218	8
1226	8
1234	8
1242	8
1250	8
1258	8
1266	8
1274	8
1282	8
1290	8
1298	8
1306	8
1314	8
1322	8
1330	8
1338	8
1346	8
1354	8
1362	8
1370	8
1378	8
1386	8
1394	8
1402	8
1410	8
1418	8
1426	8
1434	8
1442	8
1450	8
1458	8
1466	8
1474	8
1482	8
1490	8
1498	8
1506	8
1514	8
1522	8
1530	8
1538	8
1546	8
1554	8
1562	8
1570	8
1578	8
1586	8
1594	8
1602	8
1610	8
1618	8
1626	8
1634	8
1642	8
1650	8
1658	8
1666	8
1674	8
1682	8
1690	8
1698	8
1706	8
1714	8
1722	8
1730	8
1738	8
1746	8
1754	8
1762	8
1770	8
1778	8
1786	8
1794	8
1802	8
1810	8
1818	8
1826	8
1834	8
1842	8
1850	8
1858	8
1866	8
1874	8
1882	8
1890	8
1898	8
1906	8
1914	8
1922	8
1930	8
1938	8
1946	8
1954	8
1962	8
1970	8
1978	8
1986	8
1994	8
2002	8

640 / 8 = 80



FCFS		磁盘调度的次序以及平均寻道长度					
被访问的 下一个磁 道号	移动的磁 道数	SSTF		SCAN		CSCAN	
		被访问的 下一个磁 道号	移动的 磁道数	被访问的 下一个磁 道号	移动的 磁道数	被访问的 下一个磁 道号	移动的 磁道数
98	45	65	12	37	16	14	39
183	85	67	2	14	23	37	23
37	146	37	30	65	51	65	28
122	85	14	23	67	2	67	2
14	108	98	84	98	31	98	31
124	110	122	24	122	24	122	24
65	59	124	2	124	2	124	124
67	2	183	59	183	59	183	183
平均寻道长度 80		平均寻道长度 29.5		平均寻道长度 26		平均寻道长度 26	

2.

(1) 如果盘块地址只需用 2 个字节来描述, 则该磁盘系统中盘块的数目将小于等于 2^{16} 即 65536 块, 故文件的大小也不会超过 65536 块; 而每个盘块中可存放 256 个盘块号, 因此系统最多只要用到二次间址。实际上, 使用 1 个一次间址项和 1 个二次间址项后, 允许文件的最大长度已达 $11 \times 256 + 256 \times 256$ 块, 已经超出了该磁盘系统中实际的盘块数目。

(2) 根据题意, 该文件的最后一个字节, 即文件结束符的字节偏移量为 18000000B, 而 $18000000 / 512$ 的商为 35156, 余数为 128, 因此该文件的最后块的逻辑块号为 35157。

由于 $10 + 170 + 170 \times 170 < 35157 \leq 10 + 170 + 170 \times 170 + 170 \times 170 \times 170$, 故该文件不仅需要使
用 10 个直接地址项, 还需要使用一次、二次及三次间址项。又因为 $35157 - (10 + 170 + 170 \times 170) = 6077$, $6077 / (170 \times 170)$ 得到商为 0, 余数为 6076, 得知该文件在三次间址时还需要 1
个二次间址块。而余数 6077 / 170 得到商为 35, 可知该文件在三次间址时还需要 36 个一次间
址块。因此该文件需要数据块: $35157 + (1 + 1 + 170 + 1 + 1 + 36) = 35367$ 个。故共需 35367 个物理
块。

40. 中断和陷阱的区别 (可能不考...)

- 中断源不同
- 陷阱通常由处理机正在执行的现行指令引起的, 而中断则由与现行指令无关的中断源引起的 (内容不同)
- 陷阱处理程序提供的服务为当前进程所用, 而中断处理程序提供的服务则不是为了当前进程的 (中断处理提供者不同)
- CPU 在执行完一条指令之后, 下一条指令开始之前响应中断, 而在一条指令执行中也可以响应陷阱 (响应时间不同)
- 某些 OS 系统中, 陷阱处理程序被规定在各自的进程上下文中执行, 而中断处理程序则在系统上下文中执行

云服务

41. 云即服务 (列举)

- 基础设施即服务 IAAS
- 平台即服务 PAAS
- 软件即服务 SAAS
- 网络即服务

a. SaaS: 提供给客户的服务是运营商运行在云计算基础设施上的应用程序, 用户可以在各种设备上通过客户端界面访问, 如浏览器。消费者不需要管理或控制任何云计算基础设施, 包括网络、服务器、操作系统、存储等等;



- b. PaaS: 提供给消费者的服务是把客户采用提供的开发语言和工具 (例如Java, python, .Net等) 开发的或收购的应用程序部署到供应商的云计算基础设施上去。客户不需要管理或控制底层的云基础设施, 包括网络、服务器、操作系统、存储等, 但客户能控制部署的应用程序, 也可能控制运行应用程序的托管环境配置;
- c. IaaS: 提供给消费者的服务是对所有计算基础设施的利用, 包括处理CPU、内存、存储、网络和其它基本的计算资源, 用户能够部署和运行任意软件, 包括操作系统和应用程序。消费者不管理或控制任何云计算基础设施, 但能控制操作系统的选择、存储空间、部署的应用, 也有可能获得有限制的网络组件 (例如路由器、防火墙、负载均衡器等) 的控制。

操作系统未来发展

如今计算机网络发展迅速, 大部分安全、可靠的操作系统仍被大多数用户喜爱和需求。将来, 可能一些操作系统将逐渐变得更加专业和小型化, 并具有以下新功能: 开源、专业、小型或微型、便携、网络、安全和可靠等特点让每个用户都能快速上手操作, 更好的满足他人和自己的需求。

在未来, 操作系统在基础保障的可靠技术领域仍会在很多技术和功能上的未知困难。(安全可靠技术的发展仍要面临巨大的挑战和困难)。

~~相信~~ 操作系统也需不断改良、创新, 相信未来会变得更强。

